

Jack Murrow

Santa Clara, California

✉ jmurrow@scu.edu • in jack-murrow • github.com/jackprogramsjp

Research Interests

Programming Languages, Programming Language Semantics, Type Systems, Formal Verification, Proof Assistants, Compiler Construction, Domain-Specific Languages

Education

Santa Clara University

BSc Computer Science and Engineering

Santa Clara, CA

2024–2028

Relevant Coursework (grades): Programming Languages (A), Discrete Mathematics (A), OO Programming and Advanced Data Structures (A), Introduction to Embedded Systems (A)

Research Experience

Santa Clara University

Undergraduate Researcher — Advisor: Prof. Krishna Ramamoorthy

Santa Clara, CA

Apr 2026–Present

Research Project: Domain-Specific Language for Graph Algorithms

- Designing a domain-specific language that combines functional and imperative programming for expressing graph algorithms, including the compiler pipeline, core calculus, and type system.
- Formalizing the language semantics in Rocq, with the goal of verifying semantics-preserving scheduling and compiler optimizations.

Professional Experience

RooHa

Software Engineering Contractor

Santa Clara, CA (Virtual)

Nov 2025–Jan 2026

- Built and shipped core features for an early-stage mobile app, designing the frontend and implementing the Firebase backend.

Udemy

Independent Instructor

Remote

May 2023–Present

- Published a free **Introductory Compiler Design** [Udemy ↗] course and curriculum with 1,500+ students and an average 4.25+ star rating
 - Developed an open-source educational compiler project to teach fundamental compiler concepts. [GitHub ↗]
 - Introduces lexical analysis, recursive-descent parsing, abstract syntax trees, interpreters, and compiler fundamentals.
- Published a paid course and curriculum on **Elixir Functional Programming and Concurrency** [Udemy ↗] with 110+ students and an average 4.50+ star rating
 - Introduces higher-order functions, partially applied functions, and tail-call recursion.
 - Introduces concurrency and a functional approach to processes (form of green thread)

Freelance Multiplayer Games

Independent Game Developer

Remote

2019–Present

- Published and scaled multiple games with 2,500,000+ cumulative plays with sustained monetization, engineering multiplayer features and distributed backend systems.

Projects

Minimal Untyped Lambda Calculus - Java Course Assignment

Remote

CSEN 171 - Design/Implementation of Programming Languages

Fall 2026

- Designed an assignment on object-oriented AST implementation and small-step operational semantics for a minimal untyped lambda calculus in Java at the request of the instructor.

Open-Source Contributor

Remote

GitHub: *Gleam* [PRs ↗]

Summer and Fall 2026

- Contributed bug fixes and compiler improvements to Gleam (20,000+ stars), a statically-typed functional programming language with JavaScript and Erlang as compilation targets, including Erlang code generation and semantic analysis. Contributions written in Rust.

Verified Stack-Language Compiler — Independent Project

Remote

GitHub (code available upon request)

Summer and Fall 2026

- Formalized a small arithmetic source language and a stack-based target virtual machine in Rocq/Coq, defining small-step and big-step operational semantics as both inductive relations and executable functions, and proved each pair equivalent.
- Implemented a compiler from the source language to stack-machine instructions and began proving semantic preservation (compiler correctness) via a step-indexed simulation argument.

Software Foundations — Independent Study [GitHub ↗]

Remote

Logical Foundations (Vol. 1) and Programming Language Foundations (Vol. 2)

Summer 2026

- Worked through and mechanized progress and preservation for the simply typed lambda calculus in Rocq/Coq.

LLVM Ahead-of-Time Compiler [GitHub ↗]

Santa Clara, CA

Santa Clara University's ACM Winter Challenge Project

Winter 2026

- Designed and implemented a compiler for a simple version of the C programming language, written in OCaml, featuring lexical analysis, parsing, abstract syntax tree construction and optimization, semantic analysis, operator overloading, simple RAI memory management, and compilation to LLVM IR.
- Awarded **1st place (Advanced Division)** in the school's ACM Winter Challenge project competing with peers

Academic & Professional Programs

2024-2028: Santa Clara University LEAD Scholars Program—University program for first-generation students

2025: MLT Ascend—Career program for low-income and/or first-generation students

Skills

Proof assistants: Rocq/Coq

Programming Languages: OCaml, C/C++, Python, Java, Elixir, Rust

Tools: LLVM, Git

Spoken Languages: English (native), French (basic), Japanese (basic)